Yet, whatever ratio is selected, attention should be paid to ensuring that the training data set is large enough to cover all the dominant characteristic features required for reliable network training as a forecaster. The remaining data set can then be used for testing the trained network on the data samples never used in the training. For this reason, it is recommended that the non-training data set should be large enough to enable building of not only the test data set but also the ***validation data set*** to be used in the overall network evaluation.

3.5.2 Determination of Network Architecture

This is the core task in building the neural network structure optimally adapted to the specific problem the network should optimally solve. In our case it would be the optimal predictor or the optimal forecaster. This task, although being very challenging, is also the most difficult to execute because it requires from the designer much skill and practical experience. Since being a nontrivial task with a multiplicity of possible solutions, there are opinions that this work is more a kind of art than an expert's routine. The issues addressed in the following present the activities to be carried out when developing the network architecture. They include the

- determination of input nodes required
- determination of output nodes
- selection of number of hidden layers
- selection of hidden neurons
- determination of node interconnection pattern
- selection of activity function of neurons.

Determination of the required ***number of input nodes*** is a relatively easy task, because it depends predominantly on the number of independent variables presented in the data set prepared. As a rule, each independent variable should be represented by its own input node. In the case of input data prepared for forecasting, the number of input nodes is directly determined by the number of ***lagged values*** to be used for forecasting of the next value

$$x(t+1) = f\,[x(t), x(t-1), x(t-2), \ldots, x(t-n)],$$

as represented in Figure 3.13.

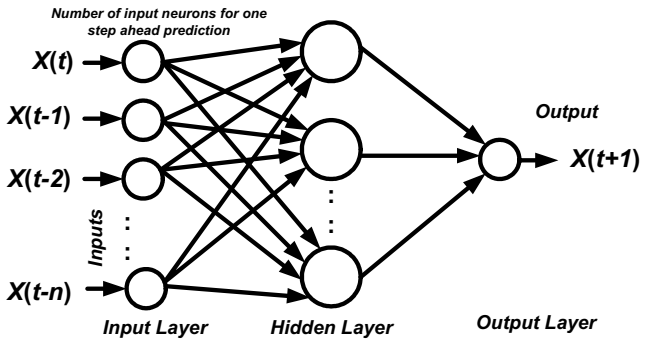


Figure 3.13. Number of input neurons for one-step-ahead forecasting

In practice, the single-step-ahead forecaster is most frequently selected because it is relatively simple and guarantees the most accurate forecasting results. Otherwise, when building a multistep predictor, the determination of the required number of input nodes is a trade-off process in the sense that (following the general inclination) this number should be selected as small as possible but so that it still guarantees good forecasting results, and as large as needed for the extraction of all relevant characteristic features and the ***autocorrelation structure*** embedded in the training data. To solve this problem optimally, some experimental runs could be of considerable use.

The ***number of output nodes***, again, is also a problem-oriented task. In the one-step-ahead forecasting it is apparent that only one output node is sufficient as the forecasting node. Correspondingly, in the case of multistep-ahead forecasting, the number of output nodes should correspond to the forecasting horizon, *i.e.* to the number of forecasts to be simultaneously presented at the network output. Alternatively, a single output node can be used and all the future forecasts required determined in the iterative steps.

In most forecasting applications, only one ***hidden layer*** is used, although some aberrations are exceptionally needed. The sufficiency of a single layer is covered by the ***Kolmogorov's superposition theorem***, which states that any continuous function $f(x)$ – which can also be an n-dimensional vector function $f(x_1, x_2, ..., x_n)$ – defined on a closed n-dimensional cube, say $[0,1]^n$, can be represented as

$$f(x_1, x_2, ..., x_n) = \sum_{i=1}^{2n+1} \psi_i \left(\sum_{j=1}^{n} \varphi_{ji}(x_j) \right),$$

where ψ_i and φ_{ji} are continuous, single-variable functions. The functions ψ_i depend on the function to be approximated f and the functions φ_{ji} are monotonously increasing functions fixed for a given n.

The theorem, as originally formulated by Kolmogorov, is an existence theorem that does not suggest any particular function to be used for approximation of a

given mapping, so that its relevancy to neural networks was not directly evident. There were even opposite views to the relevance: one opposing the relevancy (Girosi and Poggio.1989) and another in favour of it. However, it was the refinement of the theorem by Sprecher (1965) that motivated Hecht-Nielsen (1987b) to point out this reliance. He also proposed that the *k*th processing elements of the hidden layer should have the activation function

$$z_k = \sum_{i=1}^{n} \lambda^k \varphi(x_i + \varepsilon k) + k \,,$$

where the real constant λ and the monotonously increasing real continuous function depend on n, but are independent of f. Furthermore, the rational constant ε should satisfy the conditions of the ***Sprecher theorem*** $0 < \varepsilon < \delta$, $\delta > 0$. The activation function of the output layer units should be

$$y_j = \sum_{k=1}^{2n+1} g_j(z_k) \,,$$

where g_j are the real and continuous functions depending on φ and ε. Consequently, as it was shown (Hecht-Nielsen, 1987b), the Kolmogorov's theorem can be implemented exactly by a three-layer feedforward neural network having n input elements in the input layer, $(2n+1)$ processing elements in the hidden layer, and m processing elements in the output layer. This confirms the statement that even a single hidden-layer network is sufficient to reveal all the characteristic features present on the input nodes of the network. Introducing additional hidden layers increases the feature extraction capability of the network at the cost of the significantly extended training and operational time of the forecaster.

Lippmann (1987), in his celebrated paper on neurocomputing, stated clearly that a three-layer perceptron can form arbitrarily complex decision regions and can separate meshed classes, which means that no more than three network layers are needed in perceptron-like feedforward nets. This particularly holds for the networks with one output, as required for one-step-ahead forecasting. Cybenko (1989), finally underlined that the networks never need more than two hidden layers to solve most complex problems. Also, the investigation of neural network capabilities related to their internal structure has proven that two-hidden-layer networks are more prone to fall into bad local minima. DeVilliers and Barnard (1992) even pointed out that both the one- and two-hidden-layer networks perform similarly in all other respects. This can be understood from the comparison of complexity degree of two investigated networks measured by the ***Vapmik-Chervonenkis dimension***, as was done by Baum and Hausler (1989).

We now turn to the problem of the ***number of hidden neurons*** placed within the hidden layer. To determine the optimal number of hidden neurons there is no straight-forward methodology, but some rules of thumb and some suggestions how to do this have been proposed. For instance, in single-hidden-layer networks, it is recommended to take the number of hidden-layer neurons in the neighbourhood of 75% of the number of network inputs, or say between 0.5 and 3 times the number

of network inputs. The ***geometric pyramid rule***, on the other hand, suggests assigning

$$N_h = \alpha\sqrt{N_i \times N_o}\ ,$$

hidden neurons to a single hidden layer, where N_i is the number of network inputs, N_o the number of its outputs, and α is multiplication factor the value of which, depending on the complexity of the problem to be solved, should be selected in the range $0.5 < \alpha < 2$. Baum and Haussler (1989) suggested the number of neurons in the hidden layer be determined as

$$N_h \leq \frac{N_{tr} \times E_{tol}}{N_{dp} + N_o},$$

where N_{tr} is the number of training examples, E_{tol} is the error tolerance, N_{dp} is the number of data points per training example, and N_o is the number of output neurons.

Anyhow, the determination of the optimal number of hidden neurons involves trial-and-error experimentation: starting with a number of neurons within the layer to be decided – based on final accuracy of each learning process – to increase or decrease the number of hidden neurons and to start a new learning process. In this way the redundant hidden neurons can be deleted and the neurons needed for optimal performance of the layer added. Here, both starting with a relatively large or small number of neurons is possible, but starting with a large number of neurons bears the risk of long-time computation and of getting trapped in local minima.

Khorasani and Weng (1994) have presented an approach to structural adaptation of feedforward neural networks by neuron pruning, *i.e.* by addition and deletion of hidden neurons based on the activity status of individual neurons during the learning, measured by the variance of the neuron output signal and by the strength of the backpropagated error. This is a proper indication of neuron activity that helps decide which low-activity redundant neurons are to be deleted.

There is also a reliable way to determine the number of hidden neurons using the ***Akaike's information criterion*** (AIC), originally defined as

AIC = (-2) ln(Maximum likelihood) + 2(number of adjusted parameters).

The criterion statistically evaluates the goodness of a model by combining the evaluated mean squares error for training data and the number of parameters to be estimated. Seen otherwise, AIC combines a measure of fit and the penalty term to account for model complexity. Its potential application suitability for neural networks model building was recognized by Kurita (1990) and Fogel (1991), who reformulated the original form of the criterion (for statistically independent, normally distributed output errors with zero mean and with constant variance) as

$$\mathrm{AIC} = Nk \ln(\sigma^2) + 2K \,,$$

where N is the number of training data, k is the number of output units of the network, σ^2 is the ***maximum likelihood estimate*** of the mean square error for training data and K is the number of model parameters.

The application principle of the AIC is that, if two models have the same mean square error for a training data set, then the smaller sized model should be selected. Alternatively, from a set of possible models, the model with the smallest value of AIC is to be selected (Ishikawa and Moriyama, 1996; Anders and Korn, 1999). This, however, requests a set of models to be built and their parameter estimated before this application principle is used.

Unfortunately, direct application of the AIC to neural networks is rather circumstantial. It is, however, facilitated when using the ***network information criterion*** (NIC) of Stone (1977)

$$\mathrm{NIC} = -\frac{1}{T} \ln\left(L(\hat{w})\right) + \frac{\mathrm{tr}[BA^{-1}]}{T} \,,$$

which is a generalization of the AIC. The first term in the above expression represents the estimated maximum logarithmic likelihood. The matrices A and B are defined as

$$A \equiv -E[\nabla^2 \ln L_t]$$
$$B \equiv E[\nabla \ln L_t \nabla \ln L_t].$$

If the classes of models investigated include the true model, then it holds asymptotically that $A = B$ and

$$tr[BA^{-1}] = tr[I] = K,$$

where K is, again, the number of model parameters. In this case the NIC takes the form

$$\mathrm{NIC} = -\frac{1}{T} \ln L(\hat{w}) + \frac{K}{T} \,.$$

This is similar to the AIC, which in this transcription becomes

$$\mathrm{AIC} = -\frac{2}{T} \ln L(\hat{w}) + \frac{2K}{T} \,.$$